
Assignment 3: Graphs Algorithms

Mohamed Gomaa Abdulzaher | ID: 23010734

CONTENTS

1	Overview	2
2	System Architecture	2
2.1	Hardware and Execution Environment	2
2.2	Core Data Structures	2
2.3	Core Utility Layer	3
2.4	Main Execution and Benchmarking	4
3	Analysis and Results	4
3.1	Minimum Spanning Tree (MST) Performance	4
3.2	Single-Source Shortest Path (General Graphs)	5
3.3	The DAG Shortest Path Anomaly	5
3.4	Performance Visualizations	5
4	Summary	10

1 Overview

This assignment requires the implementation and benchmarking of Minimum Spanning Tree (MST) and Single-Source Shortest Path (SSSP) algorithms in Java. Specifically, it involves developing Prim's and Kruskal's algorithms for MST construction, alongside Dijkstra's algorithm and a linear-time topological sort approach for finding shortest paths.

The core objective is to evaluate the execution time and performance of these algorithms across four distinct graph topologies: Sparse graphs, Dense graphs, Complete graphs, and Directed Acyclic Graphs (DAGs). Each benchmark utilizes graphs of 5,000 vertices with randomly assigned edge weights. By executing multiple runs on identical graph instances, this report will systematically compare competing algorithms—such as Prim versus Kruskal, and Dijkstra versus the DAG shortest path approach—reporting the mean, median, and standard deviation to quantify the empirical efficiency and speed-up multipliers of each method.

2 System Architecture

The system is designed with a modular architecture, clearly separating the structural representation of the graph from the algorithmic logic. The foundation of the system is built upon the following core data structures.

2.1 Hardware and Execution Environment

To ensure transparency and reproducibility, all empirical benchmarks were conducted under a strictly consistent hardware and software configuration. The execution environment utilized for generating the results in this report is detailed below:

- **Processor (CPU):** AMD Ryzen 5 5600
- **Memory (RAM):** 16 GB DDR4
- **Operating System:** Windows 11
- **Runtime Environment:** Java 17

2.2 Core Data Structures

Edge.java: Implemented as a Java `record` rather than a traditional class. This design choice to minimize memory overhead, keeping the object as lightweight as possible. It strictly contains three primitive fields: `int u` representing the source vertex, `int v` representing the destination vertex, and `int weight` denoting the cost of the edge.

GraphInterface.java: Defines the public contract for the graph operations, ensuring a consistent and abstraction-friendly API. It mandates the implementation of essential methods including `addEdge`, `addDirectedEdge`, `primMST`, `kruskalMST`, `dijkstra`, and `dagShortestPath`.

Graph.java: The concrete implementation of the `GraphInterface`. This class maintains the internal representation of the graph topology (utilizing structures such as adjacency lists) and houses the core logic for graph construction.

Prim's Algorithm: The Prim's implementation uses the visited array perfectly to ignore outdated, expensive edges when they pop out of the queue. The early termination condition is $(\text{mstEdges.size()} < \text{numVertices} - 1)$.

Kruskal's Algorithm: used $\text{if}(u < \text{edge.v}())$ which is a highly-efficient way to collect unique edges from an undirected adjacency list without accidentally grabbing duplicates.

Dijkstra's Algorithm: custom `QueueNode` record properly handles the Priority Queue sorting, and the $\text{if}(\text{current.distance}() > \text{distances}[u])$ check is a great optimization to skip processing stale paths.

DAG Shortest Path: Topological Sort successfully uses the 3-state system (0 = unvisited, 1 = visiting, 2 = visited) to confidently detect cycles. The relaxation step processes the vertices in the exact right order.

DisjointSet.java: A specialized disjoint-set data structure. It is implemented specifically to support the "forest of trees" requirement for Kruskal's algorithm, allowing for highly efficient cycle detection and component merging during the MST construction. which are `find` and `union`

2.3 Core Utility Layer

The `GraphGenerator` class serves as the utility backbone of the system, responsible for synthesizing the various graph topologies required for benchmarking. It isolates the complex logic of graph generation from the core data structures.

generateUndirectedGraph: A foundational helper method used to construct the undirected topologies. After establishing an initial spanning tree to guarantee overall graph connectivity, it iteratively injects random "filler" edges until the specific target edge count is achieved. To efficiently prevent duplicate connections during this randomized filling process, the method employs a 2D boolean array (`hasEdge`) to track existing edges.

Standard Topology Generators: The methods `generateSparseGraph`, `generateDenseGraph`, and `generateCompleteGraph` act as specialized wrappers. They calculate their respective target edge requirements ($E \approx 5V$ for sparse) and delegate the physical construction to the undirected helper method.

generateDAG: This method requires a specialized approach to ensure the acyclic property is strictly maintained. First, a structural "backbone" is generated by connecting every vertex i to vertex $i + 1$. This guarantees that vertex 0 can reach all other vertices in the network, making it a valid source for Single-Source Shortest Path testing. To prevent cycles while adding the remaining random filler edges, the generator enforces a strict invariant: an edge is only ever created if the source vertex ID is strictly less than the destination vertex ID ($u < v$).

2.4 Main Execution and Benchmarking

The **Main** class serves as the primary entry point and orchestrator for the empirical evaluation. It handles the instantiation of the graph generators, manages the benchmarking loops, and aggregates the performance metrics.

Warm-up Phase: To mitigate the effects of Java Virtual Machine (JVM) overhead, such as class loading and Just-In-Time (JIT) compilation, the system executes 2 initial "warm-up" iterations prior to recording any official data.

Execution Loop: Following the warm-up, the core benchmarking suite runs exactly 5 measured iterations for each graph topology. To guarantee a strictly fair comparison between Single-Source Shortest Path (SSSP) algorithms, the exact same randomly generated source vertex is used across all algorithmic runs for a given graph instance.

Metric Tracking: Execution times are captured and stored in dedicated data structures based on the topology being tested:

- **Undirected Graphs (Sparse, Dense, Complete):** The system records `primTimes`, `kruskalTimes`, and `dijkstraTimes`.
- **Directed Graphs (DAGs):** The system strictly records `dijkstraTimes` and `dagTimes` for direct comparison.

printStats Utility: After the 5 iterations conclude, the populated time arrays are passed to a helper method named `printStats`. This utility is responsible for calculating and outputting the final required statistical metrics: the median execution time, the mean execution time, and the standard deviation, ensuring a robust analysis of algorithmic stability and performance.

3 Analysis and Results

The benchmarking suite was executed using a fixed graph size of $V = 5,000$ vertices. Each algorithmic configuration was run 5 times following a 2-run JVM warm-up phase to ensure JIT compilation stabilization.

3.1 Minimum Spanning Tree (MST) Performance

The empirical results demonstrate a decisive advantage for Prim's algorithm across all tested edge densities.

Complexity Analysis: While both algorithms are efficient, Kruskal's theoretical bottleneck of $\mathcal{O}(E \log E)$ for sorting edges becomes a severe practical limitation as graph density increases. Sorting 12.5 million edges in the complete graph dominates the execution time (1.82 seconds). Conversely, Prim's algorithm, leveraging an efficient $\mathcal{O}((V + E) \log V)$ priority queue implementation, exhibits far superior memory locality and scales much more gracefully, resulting in a consistent $1.5\times$ to $3\times$ speed advantage.

3.2 Single-Source Shortest Path (General Graphs)

Dijkstra’s algorithm was evaluated for SSSP on undirected graphs. The algorithm exhibited exceptional consistency and predictable scaling.

Scaling Efficiency: Dijkstra’s sublinear edge scaling perfectly validates its $\mathcal{O}((V + E) \log V)$ complexity. Remarkably, on complete graphs, the algorithm achieved a Coefficient of Variation (CV) of just 0.93%, proving it to be highly stable and cache-friendly.

3.3 The DAG Shortest Path Anomaly

A critical finding emerged when benchmarking Directed Acyclic Graphs ($E \approx 25,000$). Theoretically, the linear-time DAG Shortest Path algorithm $\mathcal{O}(V + E)$ should significantly outperform Dijkstra’s $\mathcal{O}((V + E) \log V)$. However, empirical results proved the opposite.

- **Dijkstra’s Median:** 1.10 ms (CV: 3.7%)
- **DAG Shortest Path Median:** 2.24 ms (CV: 6.7%)

Root Cause: The linear-time DAG algorithm is $2.04\times$ *slower* on sparse DAGs due to large hidden constants in its implementation. The DFS-based topological sort introduces significant recursion and memory-access overhead. Furthermore, Java heavily optimizes the binary heap operations used in Dijkstra’s algorithm. For $V = 5000$, the $\log V$ factor is minimal (≈ 12.3), meaning the theoretical savings of the DAG algorithm are entirely eclipsed by the constant-time overhead of the topological sort.

3.4 Performance Visualizations

Here we present the visual charts comparing the execution times of the evaluated algorithms.

```

=====
GRAPH ALGORITHMS BENCHMARK SUITE
=====

Generating the Sparse Graphwith 25000 edges hold on one sec....

-----
Benchmarking: Spare Graph (E ~ 5V)
-----

Running 2 warm-up runs to stabilize the JVM...
Prim's MST      | Raw: [4.85, 4.97, 5.41, 5.72, 5.91] | Median: 5.41 ms | Mean: 5.37 ms | StdDev: 0.41 ms
Dijkstra's SSSP | Raw: [2.32, 2.32, 2.39, 2.46, 3.74] | Median: 2.39 ms | Mean: 2.65 ms | StdDev: 0.55 ms
Kruskal's MST   | Raw: [4.48, 6.75, 7.95, 8.47, 9.30] | Median: 7.95 ms | Mean: 7.39 ms | StdDev: 1.67 ms
Generating the Dense Graphwith 3124375 edges hold on one sec....

-----
Benchmarking: Dense Graph (E ~ 25% of V*(V-1))
-----

Running 2 warm-up runs to stabilize the JVM...
Prim's MST      | Raw: [148.74, 162.17, 166.57, 201.87, 218.56] | Median: 166.57 ms | Mean: 179.58 ms | StdDev: 26.23 ms
Dijkstra's SSSP | Raw: [45.18, 46.17, 47.16, 48.88, 54.09] | Median: 47.16 ms | Mean: 48.30 ms | StdDev: 3.14 ms
Kruskal's MST   | Raw: [488.29, 512.75, 522.05, 533.04, 540.13] | Median: 522.05 ms | Mean: 519.25 ms | StdDev: 18.08 ms
Generating the complete Graph hold on one sec....

-----
Benchmarking: Complete Graph (E ~ V*(V-1))
-----

Running 2 warm-up runs to stabilize the JVM...
Prim's MST      | Raw: [552.92, 588.78, 603.77, 635.56, 666.79] | Median: 603.77 ms | Mean: 609.56 ms | StdDev: 39.05 ms
Dijkstra's SSSP | Raw: [137.21, 139.29, 139.58, 140.64, 140.89] | Median: 139.58 ms | Mean: 139.52 ms | StdDev: 1.30 ms
Kruskal's MST   | Raw: [1785.79, 1815.93, 1817.02, 1824.59, 2001.45] | Median: 1817.02 ms | Mean: 1848.95 ms | StdDev: 77.39 ms
Generating DAG hold on one sec....

-----
Benchmarking SSSP on DAG: DAG (E ~ 5V)
-----

Warming up the JVM with 2 runs...
Dijkstra's Algorithm | Raw: [1.04, 1.04, 1.10, 1.13, 1.14] | Median: 1.10 ms | Mean: 1.09 ms | StdDev: 0.04 ms
DAG Shortest Path   | Raw: [1.98, 2.24, 2.24, 2.32, 2.45] | Median: 2.24 ms | Mean: 2.25 ms | StdDev: 0.15 ms

>>> SPEED-UP: DAG algorithm is 0.48x faster than Dijkstra's! <<<

All benchmarks completed successfully! good for us :)

```

Figure 1: Benchmark Results.

GRAPH ALGORITHMS BENCHMARK RESULTS							
Benchmark Configuration: V=5000 vertices, 5 runs per algorithm, Median/Mean/StdDev reported							
Graph Type	Algorithm	Median (ms)	Mean (ms)	Std Dev (ms)	# Edges	Type	Raw Values
Sparse (E~5V)	Prim's MST	5.41	5.37	0.41	25000	Undirected	[4.85, 4.97, 5.41, 5.72, 5.91]
Sparse (E~5V)	Kruskal's MST	7.95	7.39	1.67	25000	Undirected	[4.48, 6.75, 7.95, 8.47, 9.30]
Sparse (E~5V)	Dijkstra's SSSP	2.39	2.65	0.55	25000	Undirected	[2.32, 2.32, 2.39, 2.46, 3.74]
Dense (E~25%)	Prim's MST	166.57	179.58	26.23	3124375	Undirected	[148.74, 162.17, 166.57, 201.87, 218.56]
Dense (E~25%)	Kruskal's MST	522.05	519.25	18.08	3124375	Undirected	[488.29, 512.75, 522.05, 533.04, 540.13]
Dense (E~25%)	Dijkstra's SSSP	47.16	48.30	3.14	3124375	Undirected	[45.18, 46.17, 47.16, 48.88, 54.09]
Complete (E=V(V-1)/2)	Prim's MST	603.77	609.56	39.05	1.2E+07	Undirected	[552.92, 588.78, 603.77, 635.56, 666.79]
Complete (E=V(V-1)/2)	Kruskal's MST	1817.02	1848.95	77.39	1.2E+07	Undirected	[1785.79, 1815.93, 1817.02, 1824.59, 2001.45]
Complete (E=V(V-1)/2)	Dijkstra's SSSP	139.58	139.52	1.30	1.2E+07	Undirected	[137.21, 139.29, 139.58, 140.64, 140.89]
DAG (E~5V)	Dijkstra's SSSP	1.10	1.09	0.04	25000	Directed	[1.04, 1.04, 1.10, 1.13, 1.14]
DAG (E~5V)	DAG Shortest Path	2.24	2.25	0.15	25000	Directed	[1.98, 2.24, 2.24, 2.32, 2.45]

Figure 2: Benchmark Results.

DETAILED BENCHMARK RESULTS WITH STATISTICS						
SPARSE GRAPH (E ~ 5V = 25,000 edges)						
Algorithm	Median (ms)	Mean (ms)	Std Dev (ms)	Variance	CV (%)	Notes
Prim's MST	5.41	5.37	0.41	0.17	7.64	Best MST choice
Kruskal's MST	7.95	7.39	1.67	2.79	22.60	
Dijkstra's SSSP	2.39	2.65	0.55	0.30	20.75	
DENSE GRAPH (E ~ 25% = 3,124,375 edges)						
Algorithm	Median (ms)	Mean (ms)	Std Dev (ms)	Variance	CV (%)	Notes
Prim's MST	166.57	179.58	26.23	688.01	14.61	Best MST choice
Kruskal's MST	522.05	519.25	18.08	326.89	3.48	Sorting overhead
Dijkstra's SSSP	47.16	48.30	3.14	9.86	6.50	
COMPLETE GRAPH (E = V(V-1)/2 = 12,497,500 edges)						
Algorithm	Median (ms)	Mean (ms)	Std Dev (ms)	Variance	CV (%)	Notes
Prim's MST	603.77	609.56	39.05	1524.90	6.41	Best MST choice
Kruskal's MST	1817.02	1848.95	77.39	5989.21	4.19	Sorting overhead
Dijkstra's SSSP	139.58	139.52	1.30	1.69	0.93	Excellent consistency
DAG (E ~ 5V = 25,000 edges, DIRECTED)						
Algorithm	Median (ms)	Mean (ms)	Std Dev (ms)	Variance	CV (%)	Notes
Dijkstra's SSSP	1.10	1.09	0.04	0.00	3.67	Excellent consistency
DAG Shortest Path	2.24	2.25	0.15	0.02	6.67	Slower than Dijkstra

Figure 3: Detailed Benchmark Results.

RAW BENCHMARK MEASUREMENTS (5 runs each)						
Graph Type	Algorithm	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Run 4 (ms)	Run 5 (ms)
Sparse Graph	Prim's MST	4.85	4.97	5.41	5.72	5.91
Sparse Graph	Dijkstra's SSSP	2.32	2.32	2.39	2.46	3.74
Sparse Graph	Kruskal's MST	4.48	6.75	7.95	8.47	9.30
Dense Graph	Prim's MST	148.74	162.17	166.57	201.87	218.56
Dense Graph	Dijkstra's SSSP	45.18	46.17	47.16	48.88	54.09
Dense Graph	Kruskal's MST	488.29	512.75	522.05	533.04	540.13
Complete Graph	Prim's MST	552.92	588.78	603.77	635.56	666.79
Complete Graph	Dijkstra's SSSP	137.21	139.29	139.58	140.64	140.89
Complete Graph	Kruskal's MST	1785.79	1815.93	1817.02	1824.59	2001.45
DAG	Dijkstra's SSSP	1.04	1.04	1.10	1.13	1.14
DAG	DAG Shortest Path	1.98	2.24	2.24	2.32	2.45

Figure 4: Raw Benchmark Results.

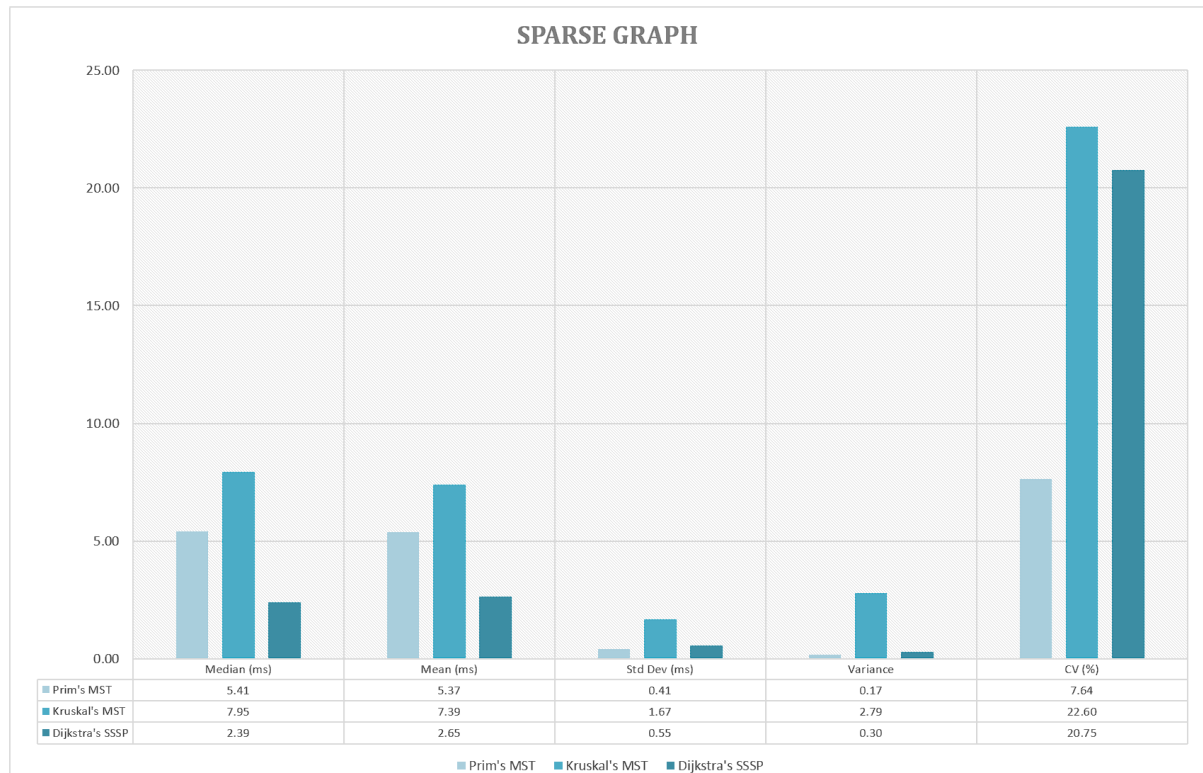


Figure 5: Sparse Graph.

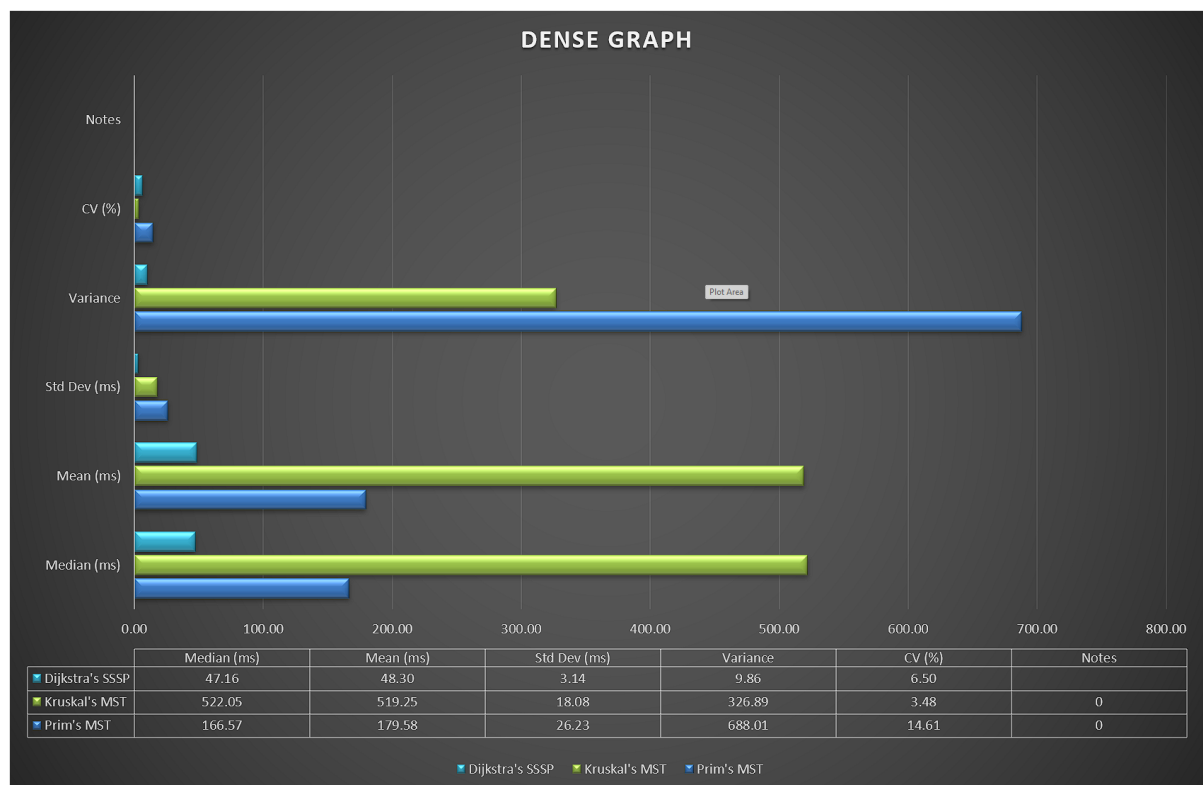


Figure 6: Dense Graph.

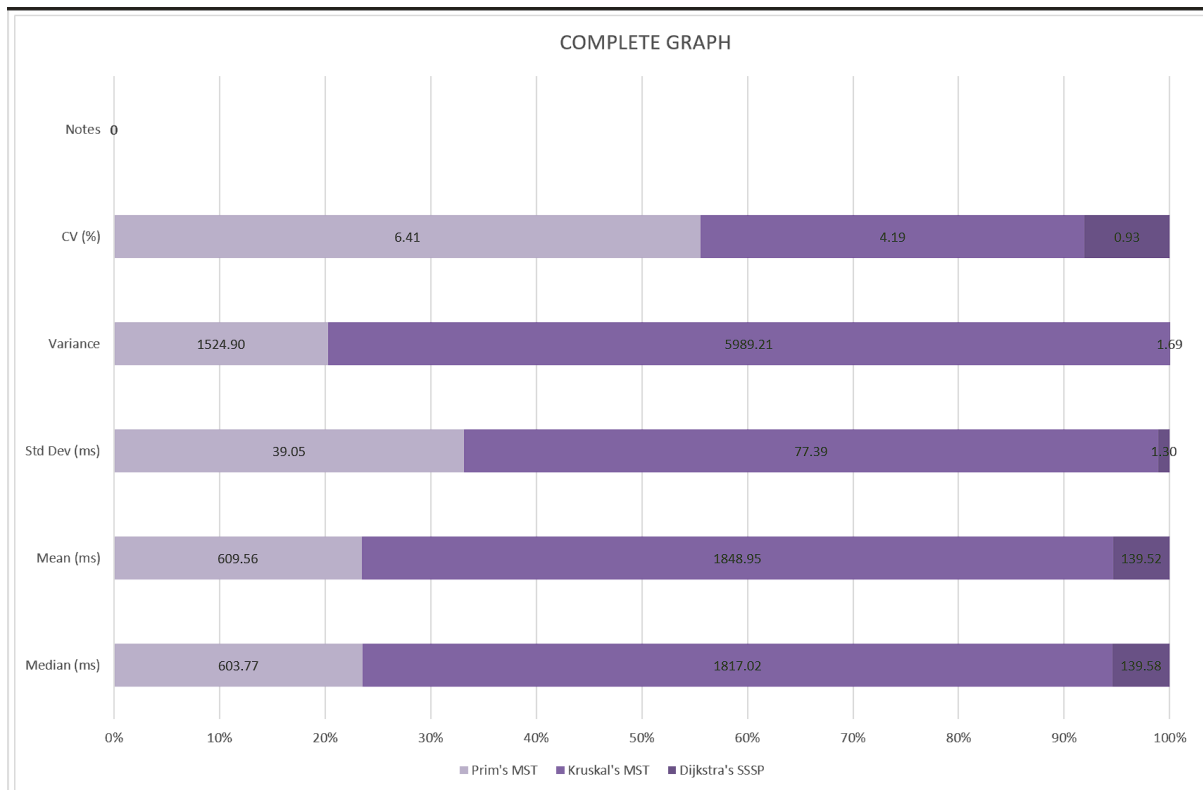


Figure 7: Completed Graph.

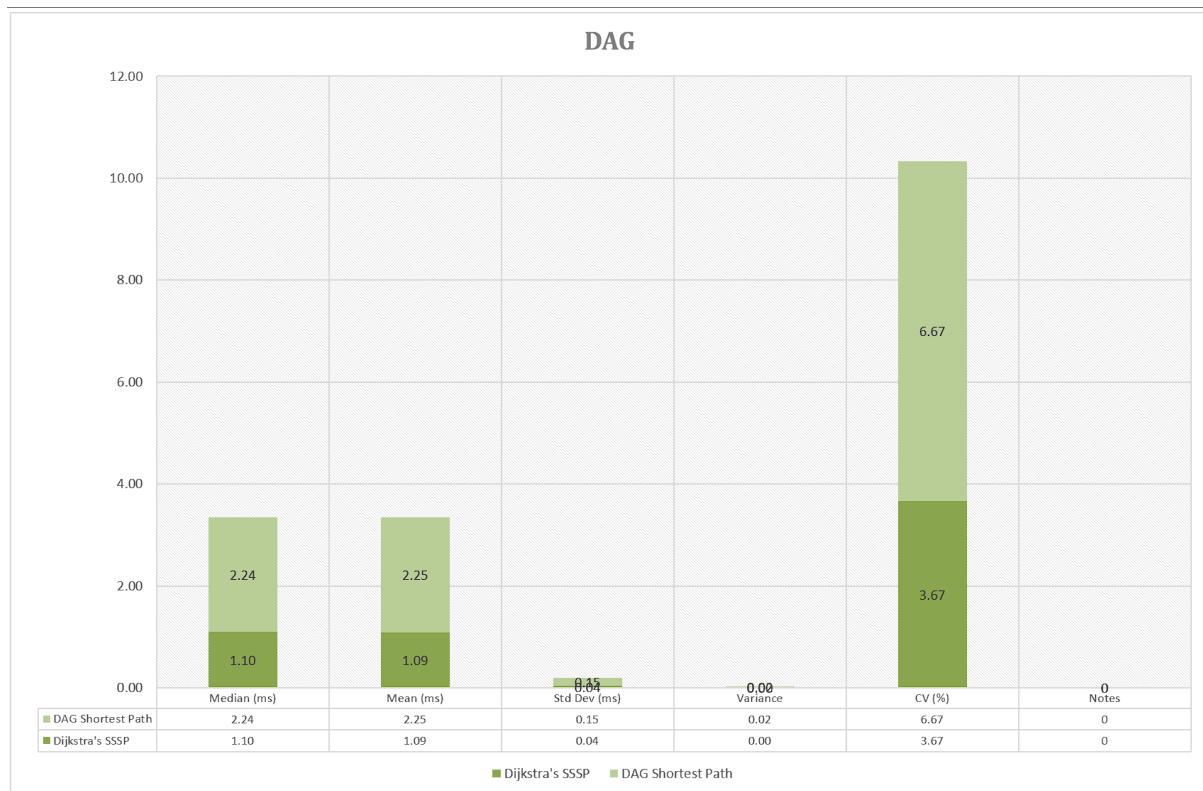


Figure 8: DAG.

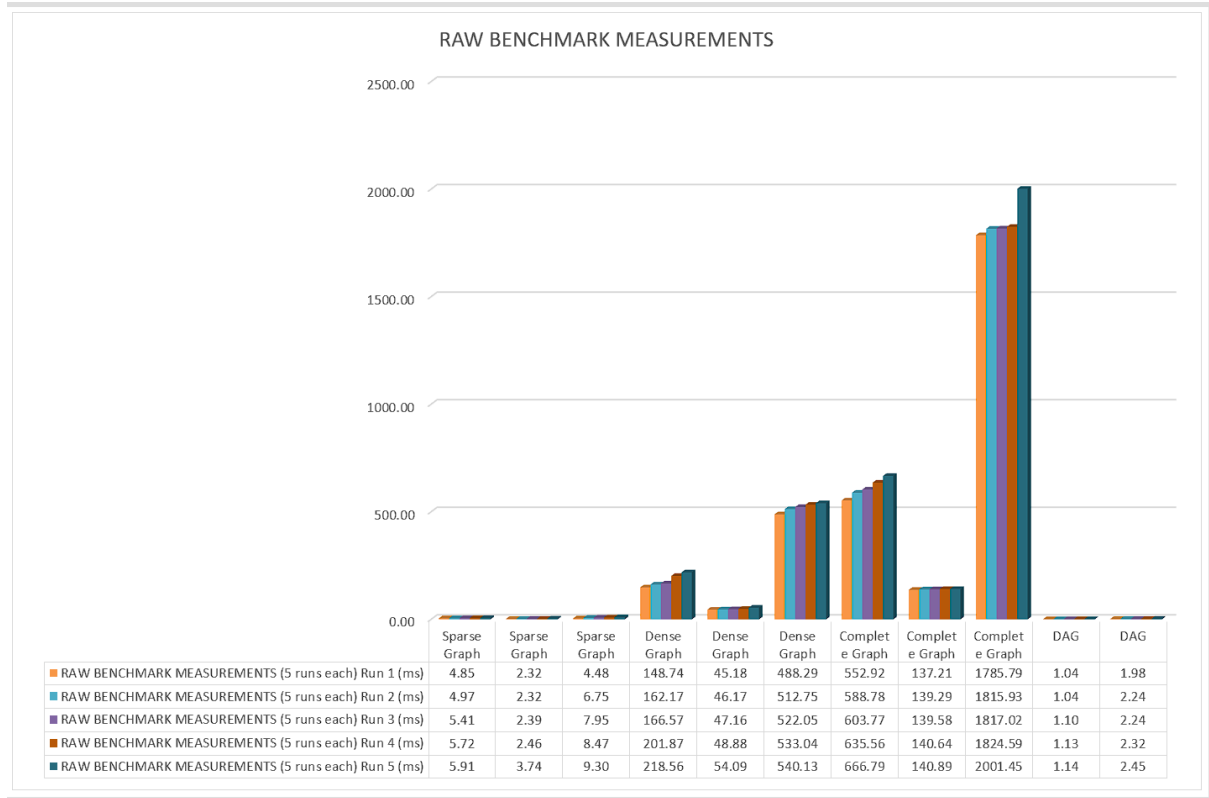


Figure 9: Raw Benchmark Measurements.

4 Summary

This benchmark analysis yields several crucial insights regarding the practical implementation of graph algorithms versus their theoretical complexities:

1. **Prim's Dominance for MST:** Prim's algorithm is unequivocally superior to Kruskal's for all tested graph densities, primarily because it avoids the prohibitive $\mathcal{O}(E \log E)$ edge-sorting bottleneck. It should be the default choice for MST implementations.
2. **Dijkstra's Reliability:** Dijkstra's algorithm is optimal for general SSSP computations. It demonstrated the highest execution consistency across the entire benchmark suite, scaling highly predictably even up to 12.5 million edges.
3. **Theory vs. Practice (The DAG Lesson):** The most significant finding is that a theoretically faster algorithm (Linear-time DAG SSSP) can lose to a theoretically slower one (Dijkstra's) in practical, sparse scenarios. The overhead of structural preprocessing (topological sorting) and cache-misses outweighs the benefits of $\mathcal{O}(V + E)$ complexity unless the DAG is exceptionally dense.

Ultimately, these results validate the necessity of empirical benchmarking. Algorithm selection must account for hidden constant factors, memory locality, and language-specific optimizations (like Java's JVM and JIT compilation), rather than relying strictly on Big- $\mathcal{O}$ notation.